1. Introduction

Course overview (1/2)

일차		구분	세부 내용	실습
1일차	오전	1. Introduction	Course Intro	
		2. Basic architecture	DNN	
			CNN and variants	ResNet
	오후	3. Attention	Transformer	
			Vision Transformer and variants	ViT
2일차	오전	4. Applications	Detection	DETR
			Tracking	
	오후		Segmentation	U-net
		5. Generative models	Latent representation, GAN	
			Diffusion	DDPM
			Text-to-Image, Latent diffusion	Stable diffusion
		6. Closing	Course review	

Course overview (2/2)

- **Lecture materials**
  - Contents
    - Technology trend & concepts
    - In-depth study on key papers
  - English
    - To convey meanings clearly and precisely
  - Math & Equations
    - Only conceptual explanation
    - Better understanding on the underlying meaning

- **Practice materials**
  - Implementing 4 out of 6 key papers
  - Jupyter notebook (Pytorch)
    - Explanation in markdown and comments
    - Feel free to make full use of generative AIs (e.g., Gemini, ChatGPT, etc.)

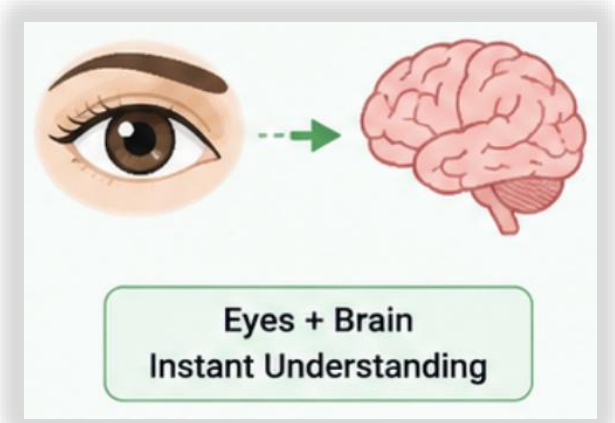
- **Test**
  - Code in the practice material

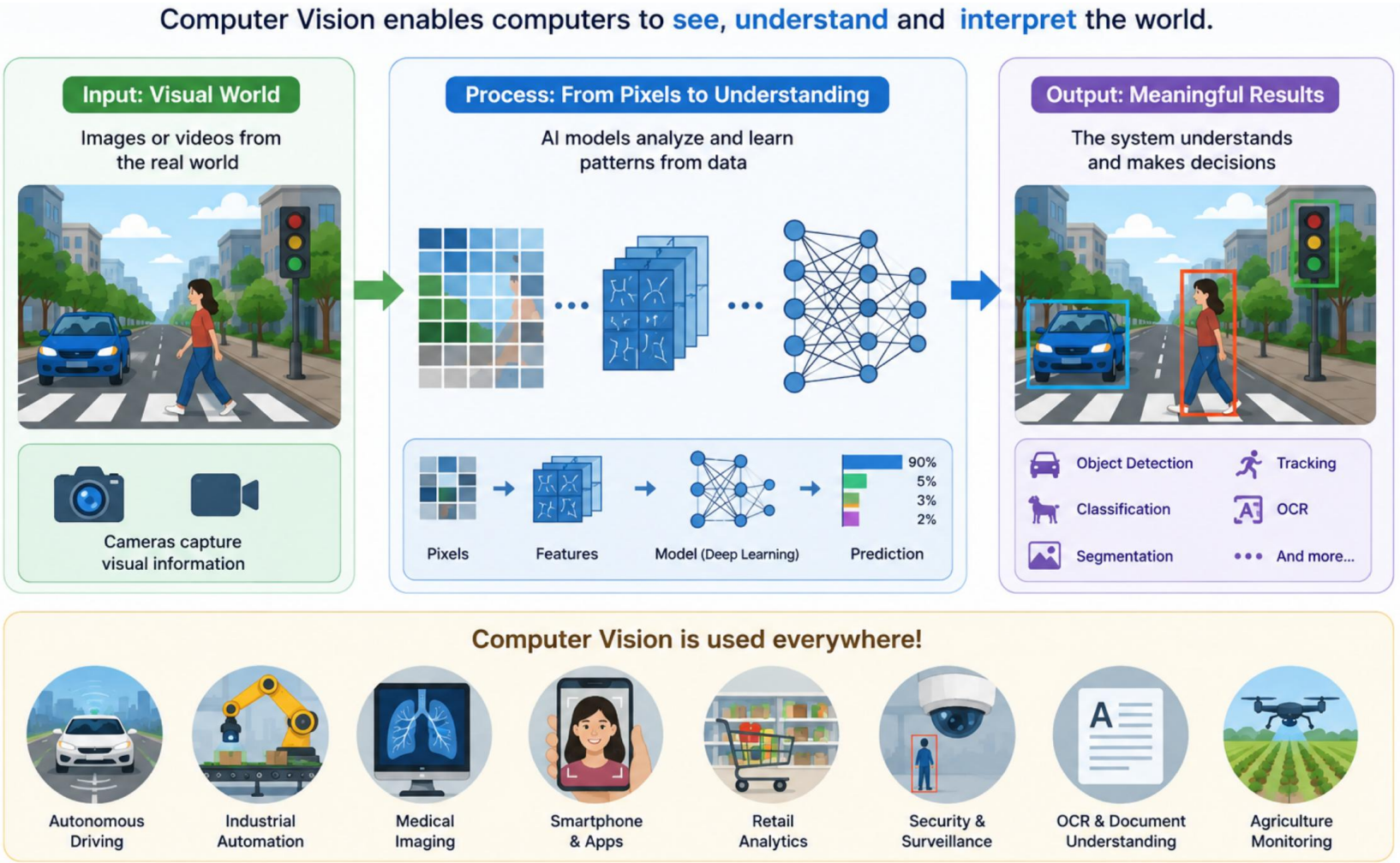
The Evolution of Computer Vision

Computer vision

- **CV (Computer Vision)** is a field of AI that enables computers to
  - see
  - understand
  - analyze
  - ***interpret***

visual information from images or videos





The early era of computer vision (1960 ~ 2010)

https://en.wikipedia.org/wiki/Sobel_operator
https://en.wikipedia.org/wiki/Canny_edge_detector
https://gaussian37.github.io/vision-concept-optical_flow/
http://www.sanko-shoko.net/note.php?id=dn5k
https://learnopencv.com/support-vector-machines-svm/
https://medium.datadriveninvestor.com/haar-cascade-classifiers-237c9193746b

- **Feature engineering era**
  - Handcrafted features (e.g., edges, corners, etc.)

Early attempts → **Pattern recognition and feature engineering** → **Machine learning-based Vision**

- Sobel filter

$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A}$$

$$\mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A}$$

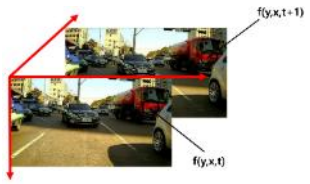


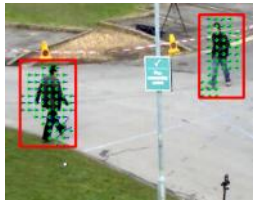
- Canny edge detector



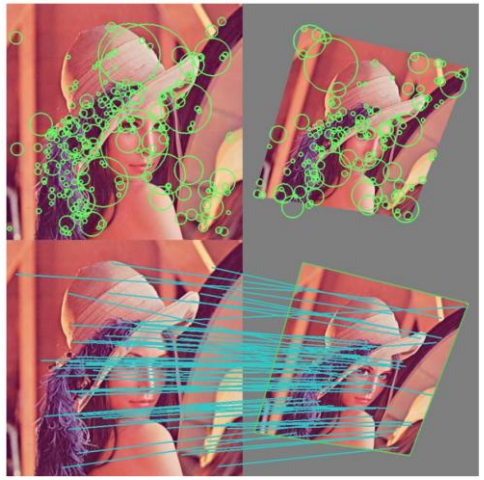


- Optical flow





- SIFT (Scale-Invariant Feature Transform)

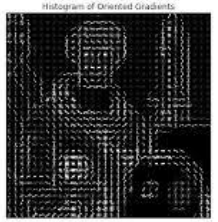


features

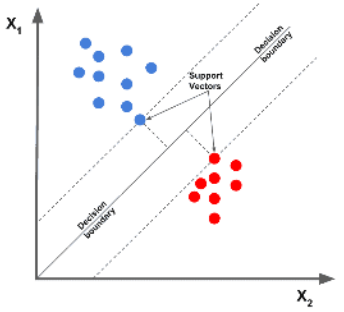
matching

- HOG (Histograms of Oriented Gradient)

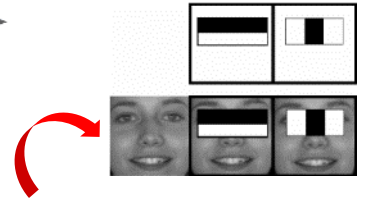


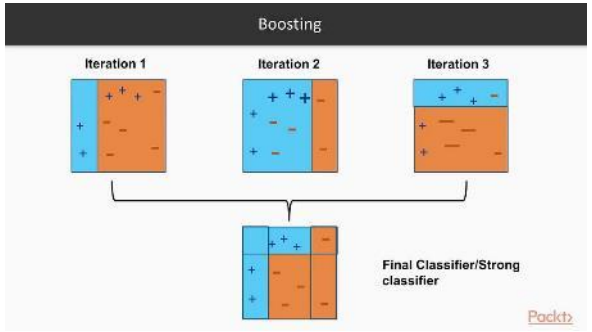


- SVM (Support Vector Machine)



- Adaboost





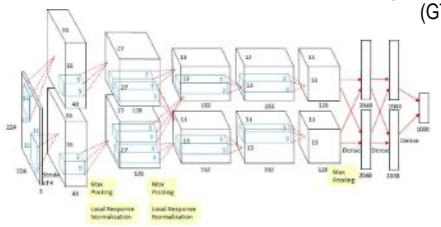
Deep learning revolution (2011 ~ 2020)

https://modulabs.co.kr/blog/alexnet_structure
https://daechu.tistory.com/10
https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/
https://dotiromoook.tistory.com/24
https://herbwood.tistory.com/15
https://modulabs.co.kr/blog/introducing-fully-convolutional-networks
https://kyujinpy.tistory.com/9
https://www.ultralytics.com/ko/blog/what-is-mask-r-cnn-and-how-does-it-work
https://lilianweng.github.io/posts/2018-08-12-vae/
https://www.linkedin.com/pulse/what-generative-adversarial-networks-gans-sushant-babbar-qpc9c

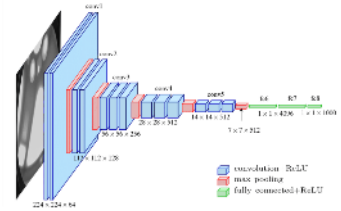
- **Paradigm shift to DNN and CNN**
  - From handcrafted features to end-to-end feature learning through deep neural networks

ImageNet breakthrough

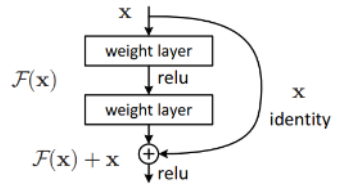
- AlexNet (Rebirth of CNN)
  - ReLu, Dropout, Max. pooling, Multi-GPUs (3GB) (GTX-580)



- VGGNet (3X3 filters)

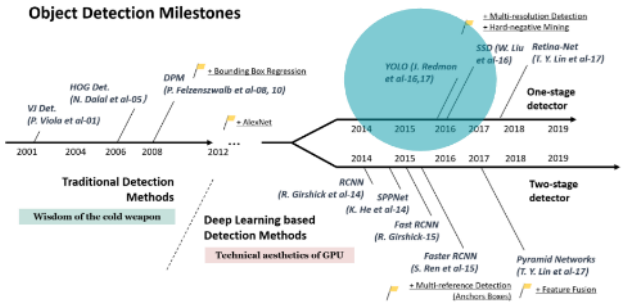


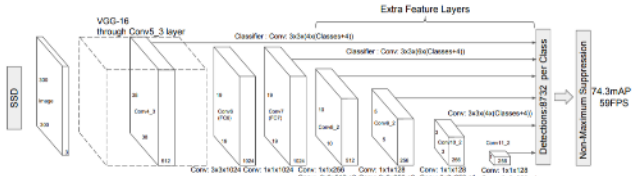
- ResNet (skip connection)



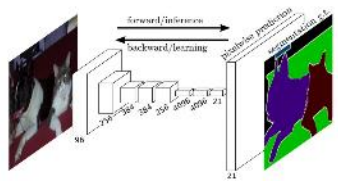
Vision applications

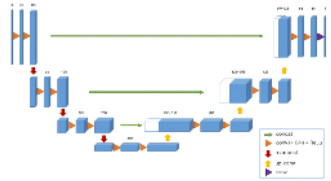
- Object detection & recognition (R-CNN, Fast R-CNN, Faster R-CNN, YOLO, SSD)

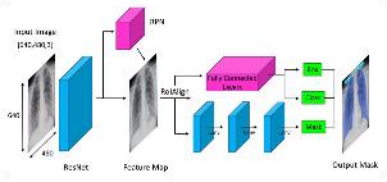




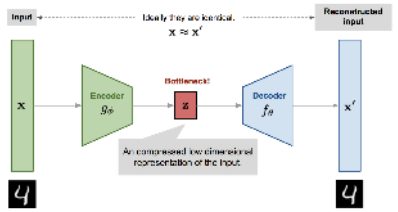
- Image segmentation & scene understanding (FCN, U-Net, Mask R-CNN, DeepLab)

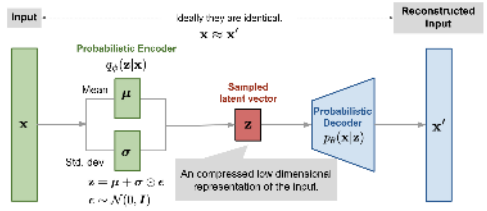


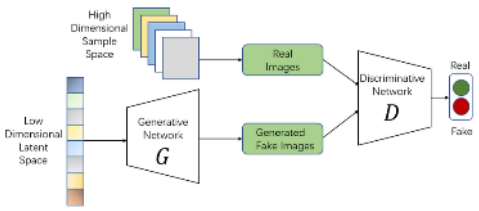




- Generative vision & representation learning (Auto Encoder, VAE, GAN)







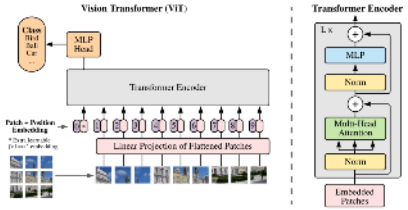
Integration with modern AI (2021 ~)

https://www.ultralytics.com/ko/blog/exploring-sam-3-meta-ais-new-segment-anything-model
https://velog.io/@rcchun/CLIP-%EB%AA%A8%EB%8D%B8-%EB%B6%84%EC%84%9D
https://sh-tsang.medium.com/review-align-scaling-up-visual-and-vision-language-representation-learning-with-noisy-text-2970ce0c4065
https://han0ahblog.tistory.com/3
https://sh-tsang.medium.com/review-dall-e-zero-shot-text-to-image-generation-f9de7a383374
https://textcortex.com/ko/post/what-is-gpt-4v-ision

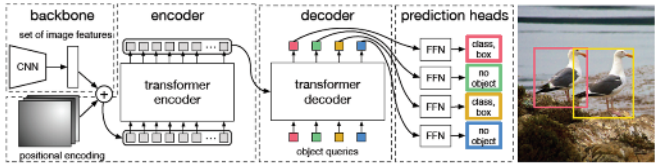
- **Multi-modal vision**
  - Understanding and generation

Transformer-based vision models

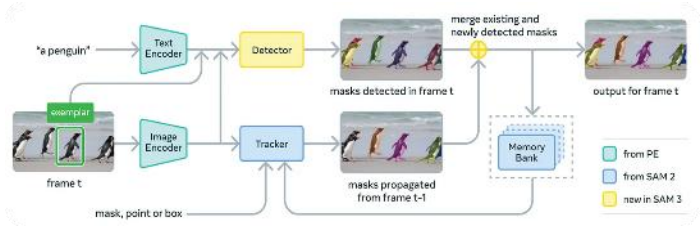
- Vision transformer (Google)



- DETR (Meta)

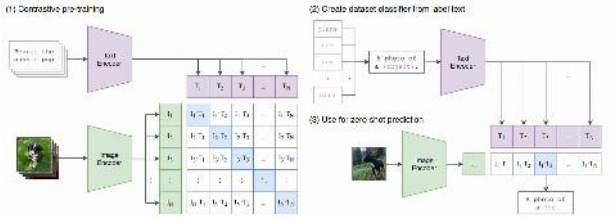


- SAM (Meta) ☞ Demo

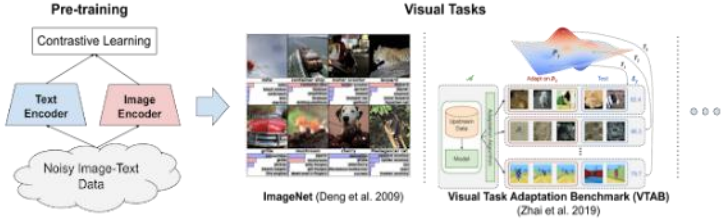


Vision-language integration

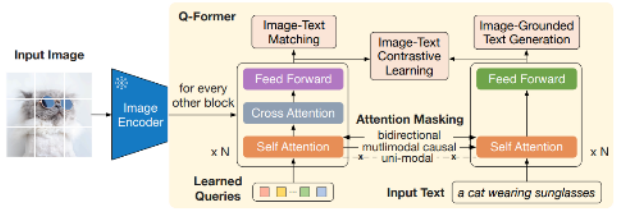
- CLIP (OpenAI)



- ALIGN (Google)



- BLIP (Salesforce) ☞ Demo



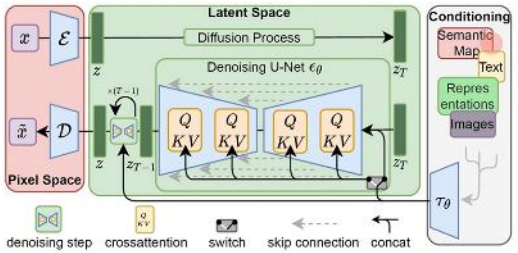
- Flamingo (DeepMind)

Generative & Interactive Vision

- DALL-E (OpenAI)



- Stable Diffusion (stability.ai) ☞ Demo



- GPT (OpenAI)



- Gemini (Google)

How vision models learn ? (1/2)

https://www.analyticsvidhya.com/blog/2021/10/human-pose-estimation-using-machine-learning-in-python/
A Simple Framework for Contrastive Learning of Visual Representations SimCLR, by Google Research, Brain Team 2020 ICML
https://medium.com/@gayatri_sharma/a-gentle-introduction-to-semi-supervised-learning-7afa5539beea
https://sanghyu.tistory.com/177
https://lilianweng.github.io/posts/2019-11-10-self-supervised/

Supervised learning

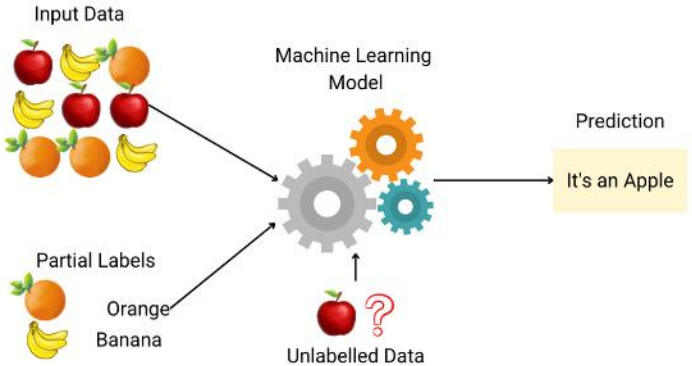
- **Classification**
  - Cat vs. Dog
  - Segmentation
    (i.e., pixel-wise classification)
- **Regression (Localization)**
  - Bounding box
    (i.e., x, y, w, h)
  - Pose estimation

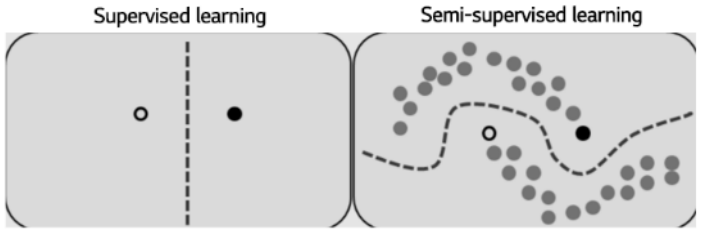




Semi-supervised learning

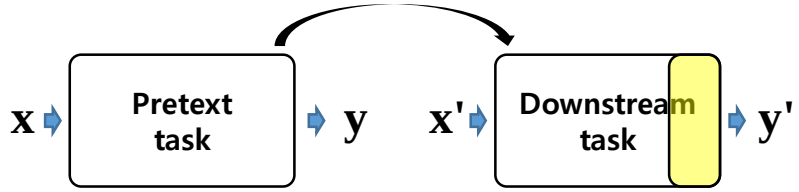
- **Few labeled** and
  Many unlabeled dataset
- **Unlabeled data**
  - Consistent network
  - Augmentation
  - Pseudo-labeling

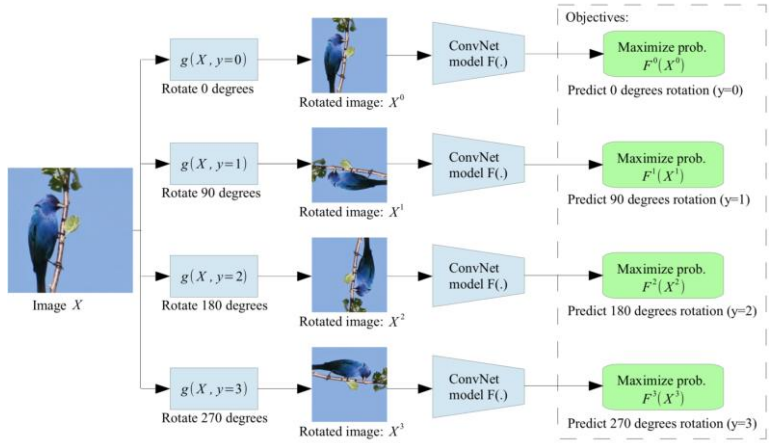




Self-supervised learning

- **Pretext task**
  - Pre-training
- **Downstream task**
  - Transfer learning
- BERT / GPT

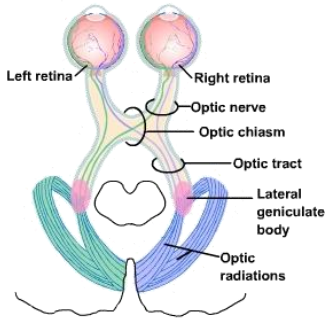
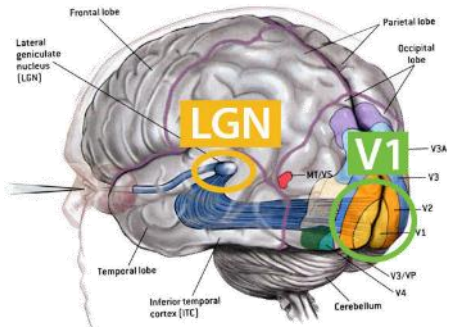
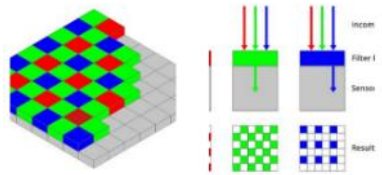
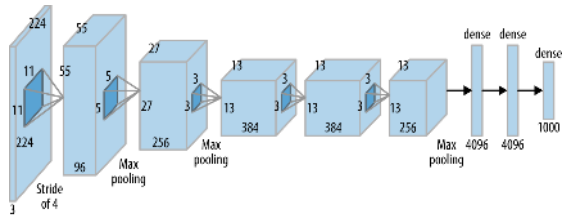




How vision models learn ? (2/2)

Category	Supervised Learning	Semi-Supervised Learning	Self-Supervised Learning
Definition	Learning with labeled data	Learning with few labeled along with large unlabeled data	Learning with no labels by generating labels from data itself
Learning Goal	Predict ground truth	Improve performance with limited labels	Learn useful data representations
Data Requirements	Large labeled dataset	Few labeled + many unlabeled dataset	Unlabeled dataset
Labeling Cost	Very high	Moderate	None
Vision Tasks	Classification Detection Segmentation	Low-label scenarios - Consistency regularization - Pseudo-labeling	Pretraining & feature learning - Pretrain on massive unlabeled image datasets - Fine-tune on specific tasks.

Human vision vs. Computer vision

Functional Stage	Human Visual System	Computer Vision System
Data Acquisition	Light captured by the retina through rods and cones.	Image captured as pixel arrays (RGB values) by camera sensors
Preprocessing & Signal Routing	LGN filters and routes visual information to cortex, organizing by color and motion	Image preprocessing (e.g., normalization, noise reduction, data augmentation)
Low-Level Feature Detection	Primary Visual Cortex (V1) detects edges, orientation, motion	Convolutional Layers in CNNs detect simple patterns (e.g., edges, textures)
Mid-Level Integration	Higher Visual Areas (V4, IT) integrate shape, color, and object identity	Deeper CNN / Transformer layers combine local features into global representations
High-Level Understanding	Prefrontal Cortex interprets visual information, linking it to memory and emotion	Fully connected layers / Vision Transformers assign semantic meaning (e.g., cat, car)
Decision & Action	Visual data informs motor cortex and decision-making (e.g., avoidance, recognition)	Visual outputs drive autonomous systems (e.g., robotics, navigation, or vision-language reasoning)
Learning & Adaptation	Learns from experience, feedback, and meaning association	Learns from large labeled datasets or reinforcement signals
Figures	 	 

https://nba.uth.tmc.edu/neuroscience/m/s2/chapter15.html
https://www.siam.org/publications/siam-news/articles/the-brain-is-a-dynamical-system/

https://www.opto-e.com/en/basics/camera-basics
https://wikidocs.net/204498

Background Knowledge

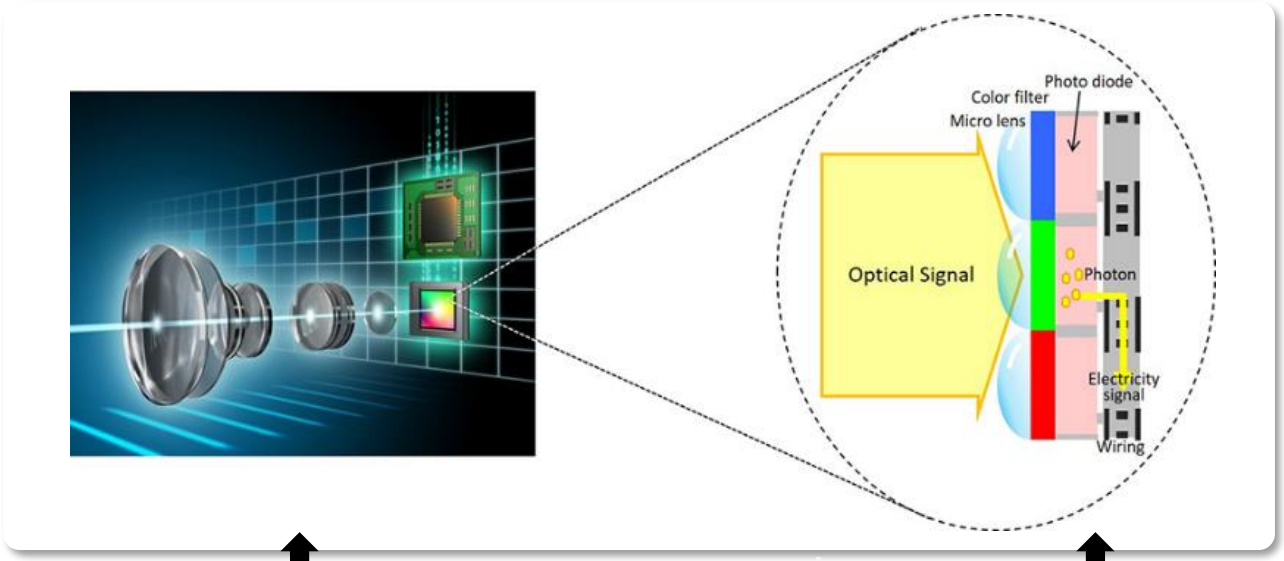
Camera image sensor

https://www.artiencegroup.com/kr/products/colorfilter/imgsensor/
https://blog.nordh.me/camera-sensor-formats/
https://www.vzense.com/blog/tech-briefs/learning-about-field-of-view-fov.html

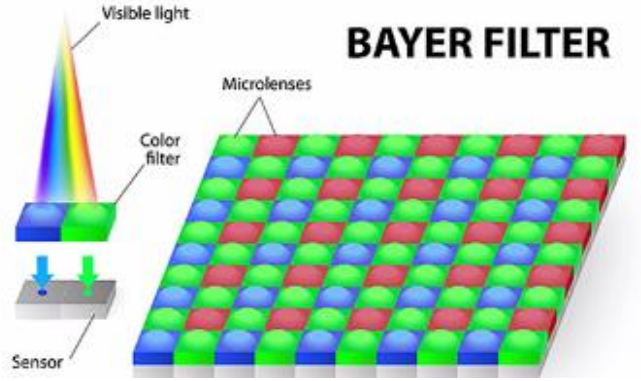
1. **Lens**
   collects light

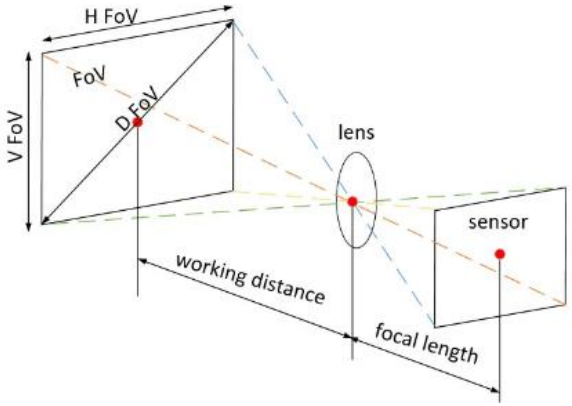
2. **Microlens**
   increases the photon collection

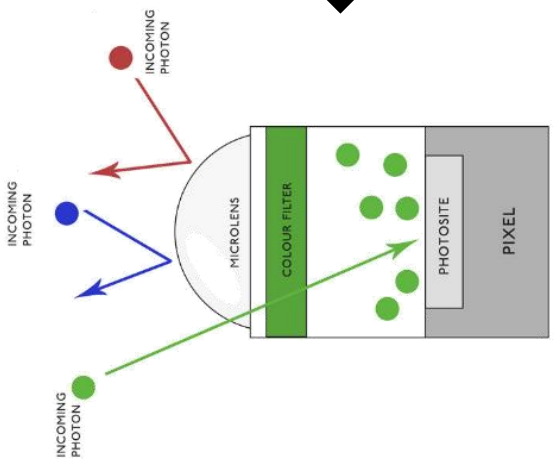
3. **Color (Bayer) filter**
   allows only certain wavelength of light



$$Y = 0.299R + 0.587G + 0.114B$$







4. **Photodiode**
   converts light into electrical current

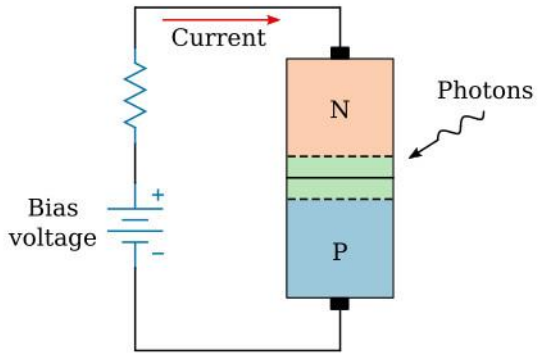
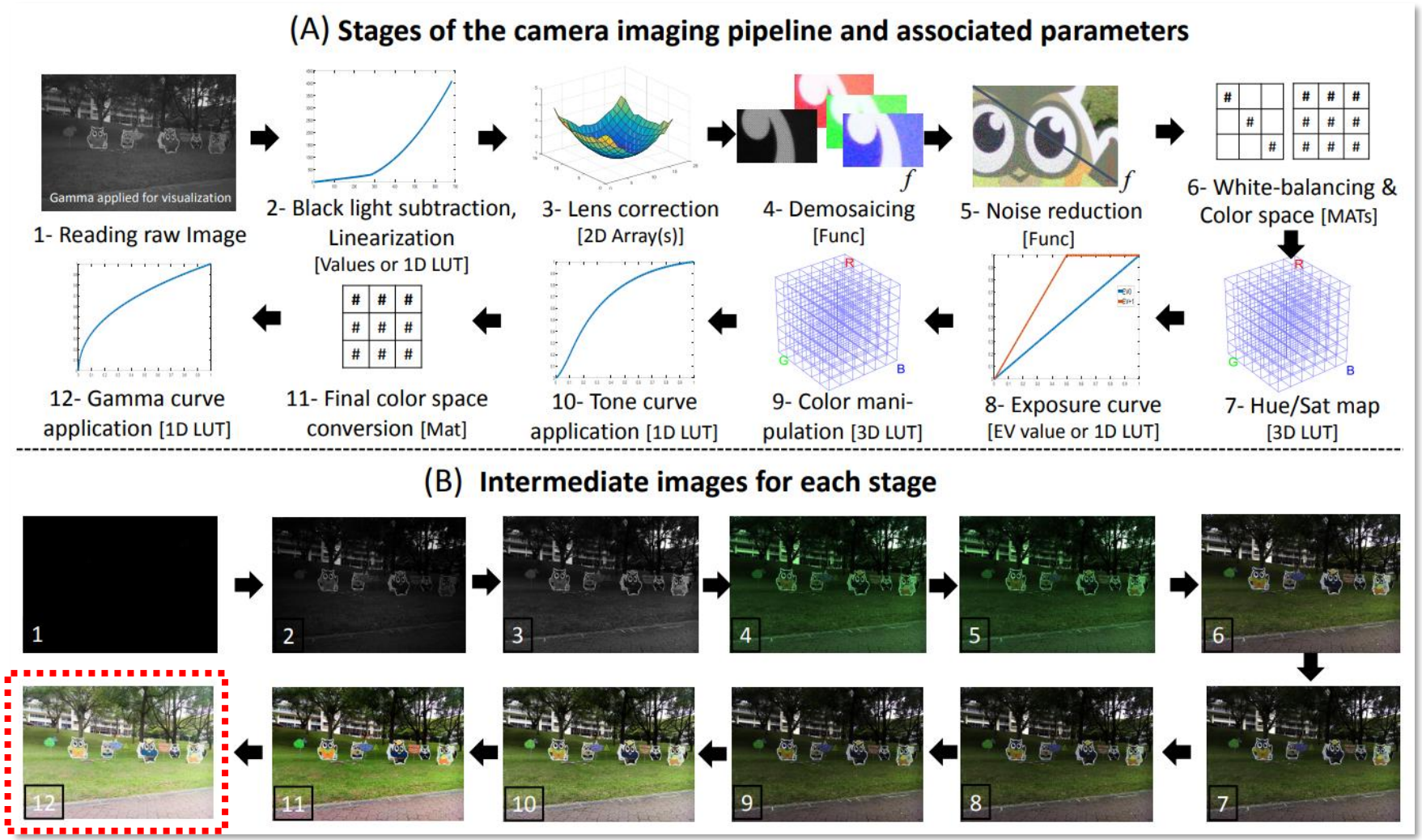


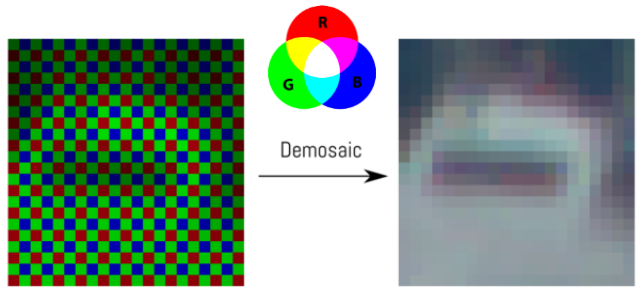
Image signal processing pipeline

https://karaimer.github.io/camera-pipeline/paper/Karaimer_Brown_ECCV16.pdf
https://yeun.github.io/2016/03/21/rgb-and-hsb.html
https://www.iphotography.com/blog/mastering-white-balance-your-key-to-vibrant-photo-colours/

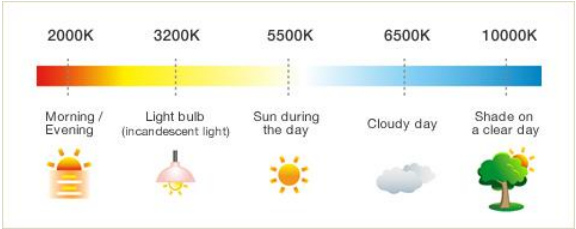
- **3A algorithm :** Auto Focus / Exposure / White balance



Demosaicing



Automatic White Balance





RGB channels

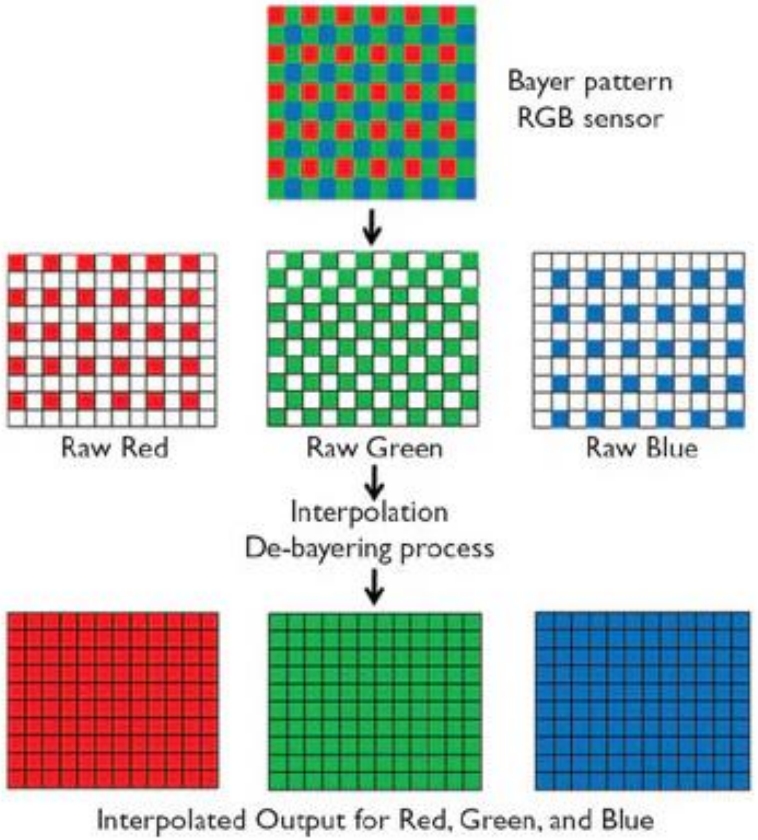
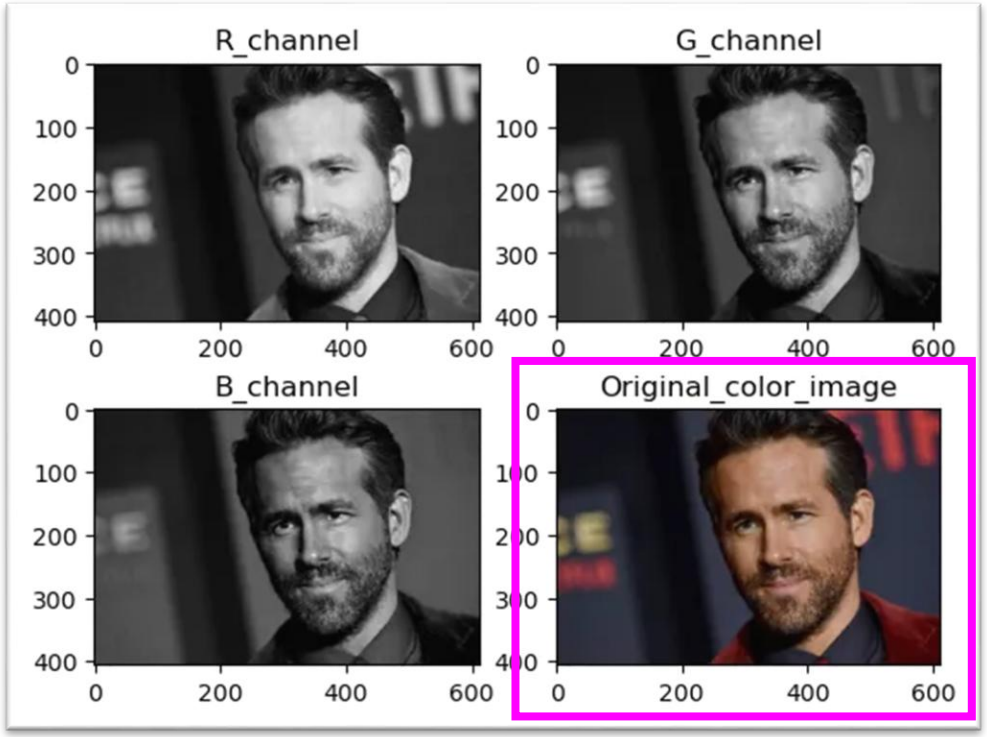


Image Signal Processing

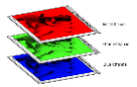
Pixel range : [0, 255]



https://www.mapir.camera/en-gb/pages/how-do-you-capture-3-channels-bands-of-separate-multispectral-reflectance-data-in-a-traditional-rgb-color-camera?srsltid=AfmBOoqSXEGikr-8GABb-0Ta757kBObHuxmGckvLArMjCcQKveSJpq0X
https://korlakuntasaikamal10.medium.com/gray-scale-and-color-images-using-opencv-eda0c13c292a

From RGB images To Mini-batch tensor

- **Deep learning models process not RGB images directly but 'numerical tensors'**

Process	What happens	Value range	Shape	Pytorch function
1. Load RGB image	Read the image file as an RGB image array 	0~255	(H, W, C)	PIL.Image.open(...).convert("RGB")
2. Convert to float	Change pixel data type from integer to floating point	0.0~255.0	(H, W, C)	transforms.ToTensor()
3. Scale pixel values	Convert pixel range from [0,255] to [0,1]	0.0~1.0	(H, W, C)	
4. Reorder axes	Change channel-last format to channel-first format		(C, H, W)	
5. Normalize the image	Apply channel-wise normalization using mean and standard deviation	Centered around 0	(C, H, W)	transforms.Normalize(mean, std)
6. Add batch dimension	Add one more dimension for batch processing		(B, C, H, W)	tensor.unsqueeze(0)

PyTorch

Vector and Matrix (1/5)

Scalar
(A single number)

$$a \in \mathbb{R}$$

Vector
(An ordered list of numbers)

$$a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \in \mathbb{R}^3$$

$$= [a_1, a_2, a_3]^T$$

Matrix
(A rectangular array of numbers)

$$C = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \\ c_{41} & c_{42} & c_{43} \end{bmatrix} \in \mathbb{R}^{4\times 3}$$

$$= [\, c^{(1)} \;\; c^{(2)} \;\; c^{(3)} \,]$$

$$= \begin{bmatrix} r^{(1)} \\ r^{(2)} \\ r^{(3)} \\ r^{(4)} \end{bmatrix}$$

	Scalar	Vector	Matrix
Geometric Representation	Magnitude	Magnitude & Direction	Structured data
Visualization	A point on a number line	An arrow in space	A collection of vectors

Identity matrix

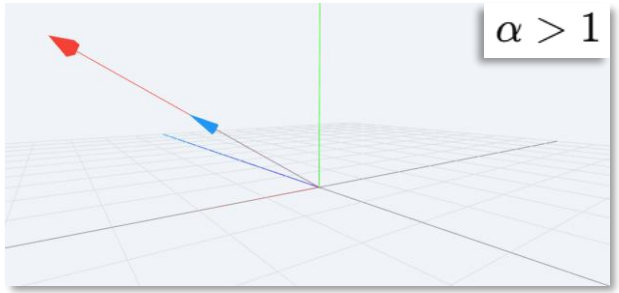
$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 3}$$

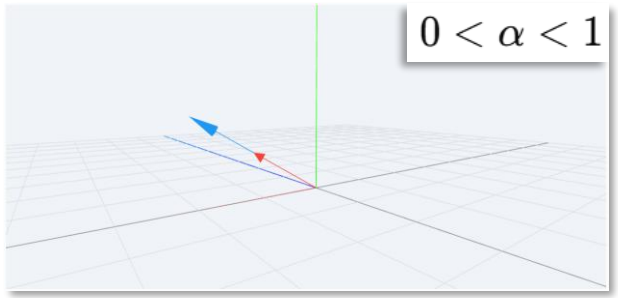
$$I \cdot A = A \cdot I = A$$

Vector and Matrix (2/5)

Scalar and Vector multiplication

$$\alpha a = \alpha \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \alpha a_1 \\ \alpha a_2 \\ \alpha a_3 \end{bmatrix} \in \mathbb{R}^3$$





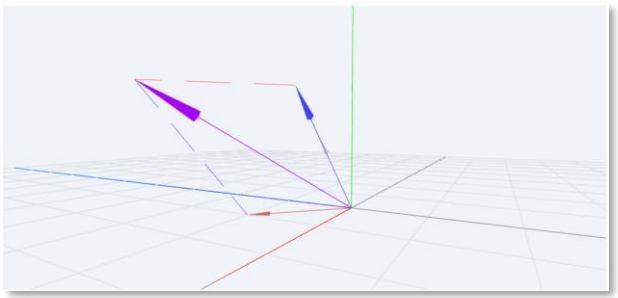
⋮

Weighted sum of two vectors

$$\alpha a + (1-\alpha) b \qquad \text{where} \quad 0 < \alpha < 1$$

$$= \alpha \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + (1-\alpha) \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$= \begin{bmatrix} \alpha a_1 + (1-\alpha) b_1 \\ \alpha a_2 + (1-\alpha) b_2 \\ \alpha a_3 + (1-\alpha) b_3 \end{bmatrix}$$



Vector and Matrix (3/5)

Matrix and Vector multiplication
(→ Column-wise weighted sum)

$$Ca = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \\ c_{41} & c_{42} & c_{43} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

$$= \begin{bmatrix} c_{11}a_1 + c_{12}a_2 + c_{13}a_3 \\ c_{21}a_1 + c_{22}a_2 + c_{23}a_3 \\ c_{31}a_1 + c_{32}a_2 + c_{33}a_3 \\ c_{41}a_1 + c_{42}a_2 + c_{43}a_3 \end{bmatrix}$$

$$= a_1 c^{(1)} + a_2 c^{(2)} + a_3 c^{(3)}$$

Vector and Matrix multiplication
(→ Row-wise weighted sum)

$$a^T C^T = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \end{bmatrix}$$

$$= \begin{bmatrix} a_1c_{11} + a_2c_{21} + a_3c_{31} & a_1c_{12} + a_2c_{22} + a_3c_{32} & a_1c_{13} + a_2c_{23} + a_3c_{33} & a_1c_{14} + a_2c_{24} + a_3c_{34} \end{bmatrix}$$

$$= a_1 r^{(1)} + a_2 r^{(2)} + a_3 r^{(3)}$$

Transpose

$$(Ca)^T = a^T C^T$$

Matrix multiplication

$$\mathbb{R}^{m \times n} = \mathbb{R}^{m \times k} \ \mathbb{R}^{k \times n}$$
$$= \mathbb{R}^{m \times i} \ \mathbb{R}^{i \times j} \ \mathbb{R}^{j \times k} \ \mathbb{R}^{k \times n}$$

Vector and Matrix (4/5)

Vector Norm
(Idea of 'distance')

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$$

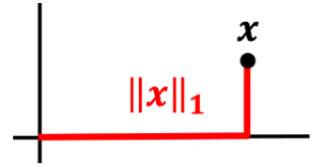
Vector norm

$$\|\mathbf{x}\| : \mathbb{R}^n \rightarrow \mathbb{R}$$

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}}$$

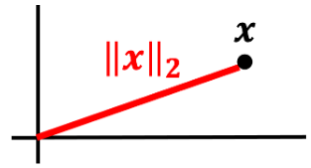
- **L1 norm**
  → Manhattan distance

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$



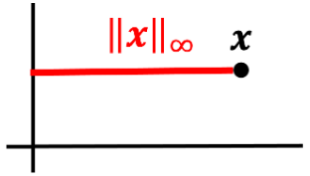
- **L2 norm**
  → Euclidean distance

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^{n} x_i^2 \right)^{\frac{1}{2}}$$



- **L∞ norm**
  → Maximum distance

$$\|\mathbf{x}\|_\infty = \max_i |x_i|$$



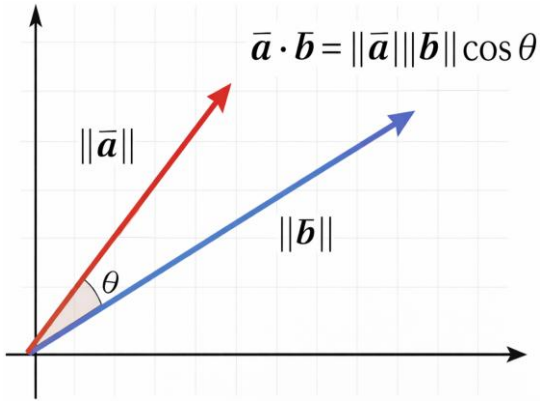
Vector and Matrix (5/5)

Inner product (i.e., Dot product)

$$a \cdot b = a^T b = a_1 b_1 + a_2 b_2 + a_3 b_3$$

$$= \|a\| \, \|b\| \cos\theta$$

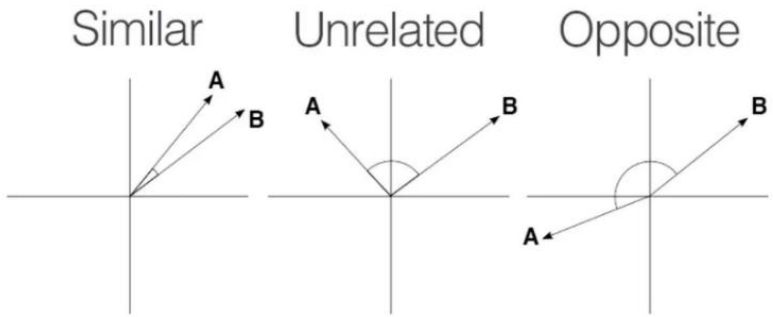
Magnitude Angle



Cosine similarity

$$\cos\theta = \frac{a \cdot b}{\|a\| \|b\|}$$

Unit vector



	Inner Product	Cosine Similarity
Aspect	Magnitude & Angle	Angle
Range	Unbounded	[-1, 1]

Math

Derivative

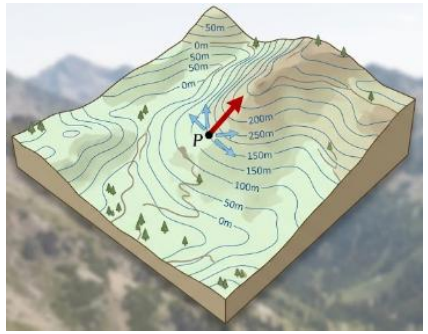
- **How fast a function changes** with respect to its input (i.e., a slope of the tangent line at an input point)

Univariate function → $y = f(x)$

↓

Derivative → $\frac{dy}{dx} = f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$

ordinary derivative



Partial derivative

- **How fast a function changes** with respect to one variable while keeping other variables fixed

Multivariate function → $f(x, y)$

↓

Partial derivative →

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y) - f(x, y)}{h}$$

$$\frac{\partial f}{\partial y} = \lim_{h \to 0} \frac{f(x, y+h) - f(x, y)}{h}$$

partial derivative

↓

Gradient ✓

→ ***"Steepest increase"*** → $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]^T$

nabla or del

Probability and Statistics (1/5)

Random variable (i.e., a function maps outcomes to real numbers)

$$X : \Omega \to \mathbb{R}$$

Ω: sample space

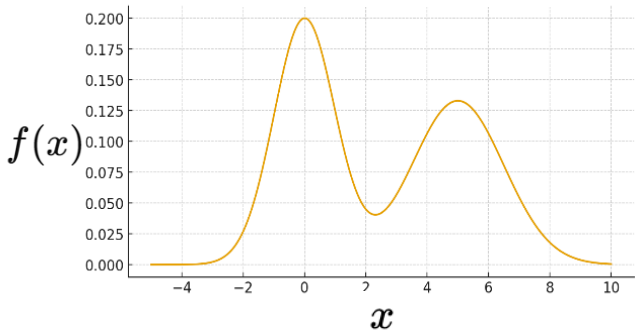
Sampling

$$x \sim X$$

PDF (Probability Density Function)

- Continuous
- Density (i.e., Probability over an interval)

$$\text{density} = \frac{\text{mass}}{\text{volume}}$$



$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, f(x)\, dx$$

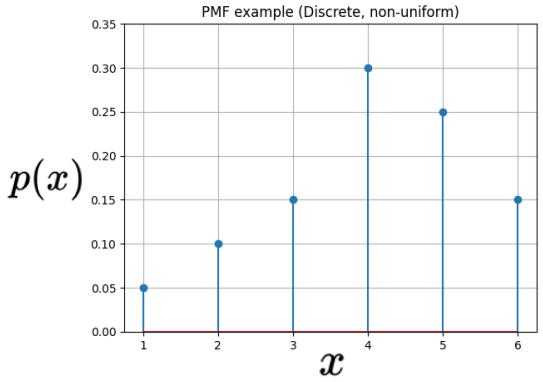
$$f(x) \geq 0$$

$$\begin{cases} P(a \leq X \leq b) = \int_a^b f(x)\, dx \\ P(X = x) = 0 \end{cases}$$

$$\int_{-\infty}^{\infty} f(x)\, dx = 1$$

PMF (Probability Mass Function)

- Discrete
- Probability



$$\mathbb{E}[X] = \sum_x x\, p(x)$$

$$0 \leq p(x) \leq 1$$

$$p(x) = P(X = x)$$

$$\sum_x p(x) = 1$$

Probability and Statistics (2/5)

Conditional probability

observation

$$p(x|y) = \frac{p(x,y)}{p(y)}$$

condition

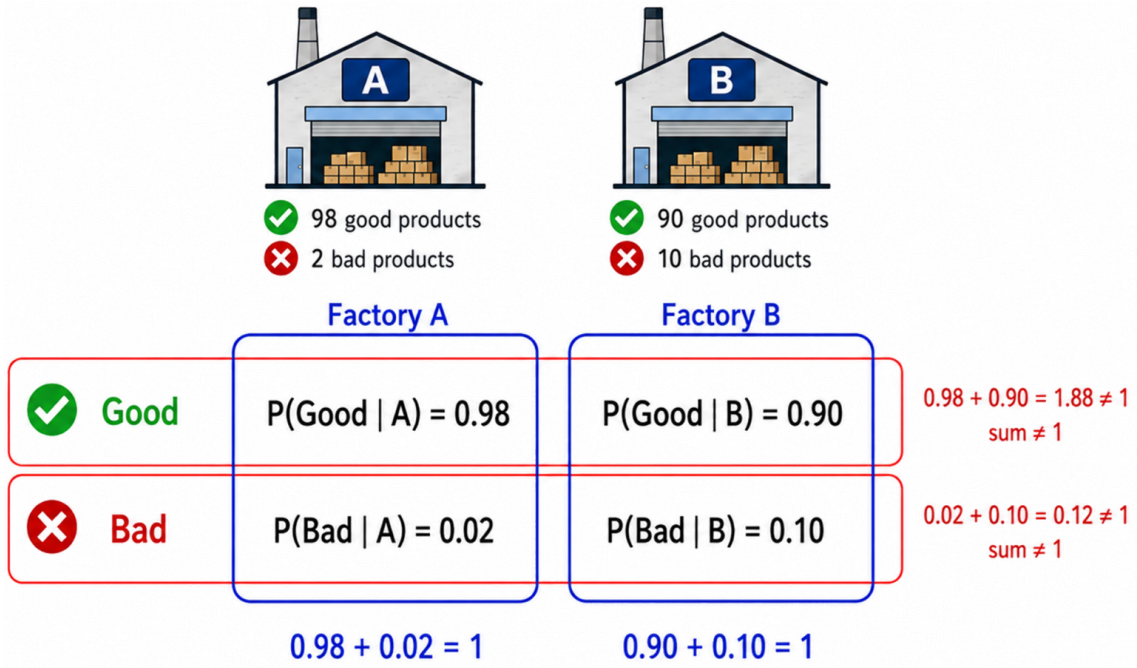
Independence

$$p(x) = p(x|y)$$
$$p(x,y) = p(x)\,p(y)$$

Conditional probability vs. Likelihood

(Not probability function)

Given condition



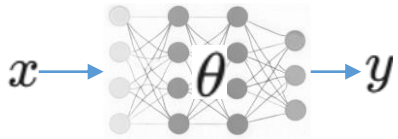
Probability and Statistics (3/5)

Bayes' rule

Posterior *Likelihood* *Prior*

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{p(D)}, \text{ where } D = \{(x_i, y_i)\}_{i=1}^{n}$$

Evidence



In neural network

$p(D \mid \theta)$

Likelihood viewpoint → **Training**

Conditional probability viewpoint → **Inference**

1. MLE (Maximum Likelihood Estimation)

$$\hat{\theta}_{MLE} = \arg\max_{\theta} p(D|\theta)$$

2. MAP (Maximum A Posteriori Estimation)

$$\hat{\theta}_{MAP} = \arg\max_{\theta} p(\theta|D)$$

$$\hat{\theta}_{MAP} = \arg\max_{\theta} \left(\log p(D|\theta) + \log p(\theta)\right)$$

$$= \arg\max_{\theta} \left(\sum_{i=1}^{n} \log p(y_i|x_i, \theta) + \log p(\theta)\right)$$

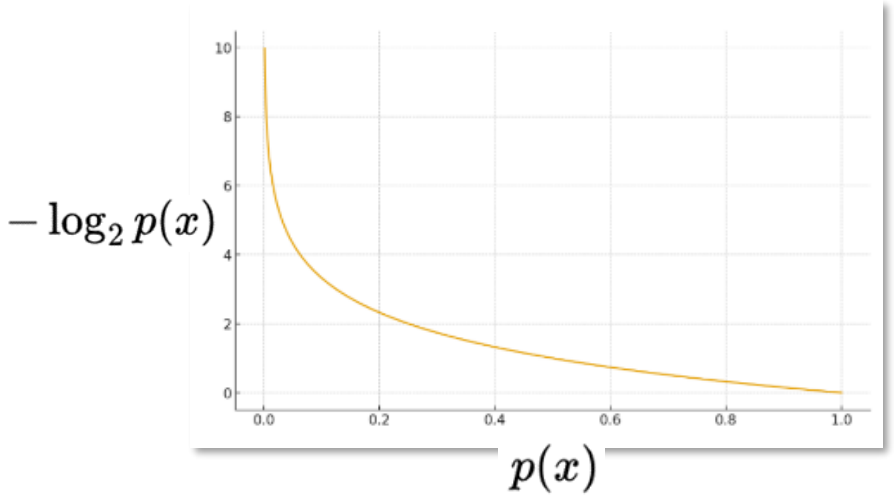
Regularization

https://www.probabilitycourse.com/chapter6/6_2_5_jensen%27s_inequality.php

Probability and Statistics (4/5)

1. Entropy (→ Uncertainty)

- The minimum average number of bits required to encode the outcomes of the variable
  - High probability → short bit length
  - Low probability → long bit length



$$H(p) = \sum_{x \in X} p(x)\,(-\log_2 p(x)) = -\sum_{x \in X} p(x) \log_2 p(x)$$

2. Cross entropy

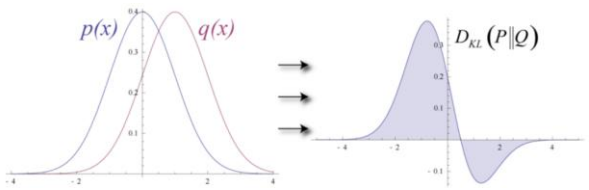
- Average coding cost of using q for data generated from p

$$H(p, q) = -\sum_{x \in X} p(x) \log_2 q(x)$$

3. KL Divergence

- A measure of how one probability distribution p differs from another reference probability distribution q

$$\begin{aligned} D_{KL}(p||q) &= H(p, q) - H(p) \\ &= -\sum_{x \in X} p(x) \log_2 q(x) + \sum_{x \in X} p(x) \log_2 p(x) \\ &= \sum_{x \in X} p(x) \log_2 \frac{p(x)}{q(x)} \geq 0 \end{aligned}$$



Probability and Statistics (5/5)

ELBO (Evidence Lower Bound)

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)}$$

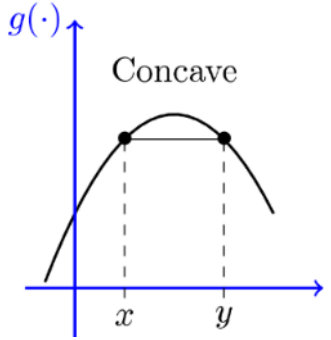
Marginalization → $p(x) = \int p(x,z)\,dz = \int p(x|z)p(z)\,dz$

$$\log p(x) = \log \int p(x,z)\,dz = \log \int q(z|x)\,\frac{p(x,z)}{q(z|x)}\,dz$$

Expectation → $= \log \mathbb{E}_{q(z|x)}\left[\frac{p(x,z)}{q(z|x)}\right]$

Jensen's inequality → $\geq \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right]$

ELBO → $\mathcal{L}(q) := \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right]$



$$= \mathbb{E}_{q(z|x)}[\log p(x|z) + \log p(z) - \log q(z|x)]$$

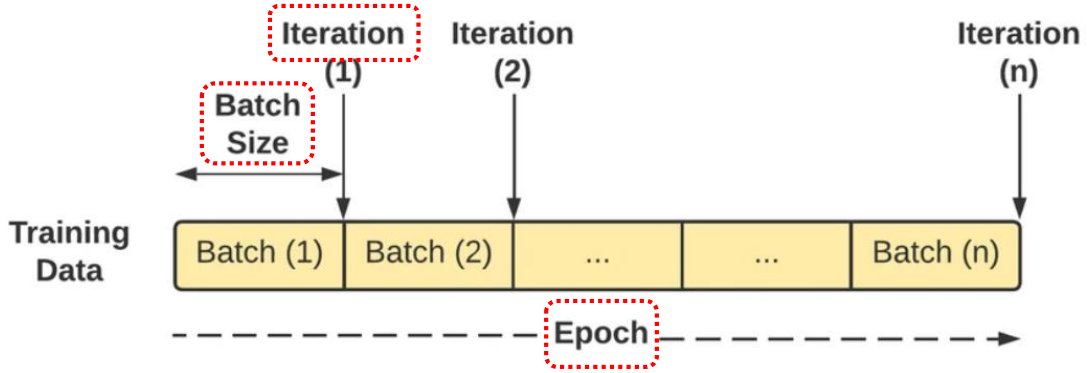
$$= \underbrace{\mathbb{E}_{q(z|x)}[\log p(x|z)]}_{\text{reconstruction term}} \uparrow - \underbrace{\mathbb{E}_{q(z|x)}\left[\log \frac{q(z|x)}{p(z)}\right]}_{D_{\mathrm{KL}}(q(z|x)\|p(z))} \downarrow$$

Training

- **Epoch**
  - One complete pass through the entire training dataset

- **Batch size**
  - The number of training examples utilized in one single forward and backward pass
    - A larger batch size provides a more stable estimate of the gradient but requires more VRAM.
    - A smaller batch size introduces "noise" that can sometimes help the model escape local minima

- **Iteration (or Step)**
  - The act of processing one batch and updating weights



$$\text{The number of Iterations (Steps) per Epoch} = \frac{\text{Total training samples}}{\text{Batch size}}$$

Image dataset

- Core Image Classification Datasets (CNN / ViT)

Dataset	Task	#Classes	#Images	Image Size	Typical Usage	Official Link
MNIST	Classification	10	70K	28×28	Toy benchmark, sanity check	http://yann.lecun.com/exdb/mnist/
Fashion-MNIST	Classification	10	70K	28×28	MNIST replacement	https://github.com/zalandoresearch/fashion-mnist
CIFAR-10	Classification	10	60K	32×32	CNN & ViT baseline	https://www.cs.toronto.edu/~kriz/cifar.html
CIFAR-100	Classification	100	60K	32×32	Fine-grained classification	https://www.cs.toronto.edu/~kriz/cifar.html
SVHN	Classification	10	600K+	32×32	Domain shift test	http://ufldl.stanford.edu/housenumbers/
STL-10	Classification / SSL	10	113K	96×96	Low-label SSL benchmark	https://cs.stanford.edu/~acoates/stl10/
Tiny ImageNet	Classification	200	100K	64×64	Lightweight ImageNet proxy	https://www.kaggle.com/c/tiny-imagenet
ImageNet-1K	Classification	1,000	1.28M	~224×224	Standard vision benchmark	https://www.image-net.org/
ImageNet-21K	Classification	21K	14M	~224×224	Large-scale pretraining	https://www.image-net.org/
Places365	Scene Classification	365	1.8M	~224×224	Scene understanding	http://places2.csail.mit.edu/
iNaturalist	Fine-grained Cls	5K+	3M+	Variable	Long-tail evaluation	https://www.inaturalist.org/

Image dataset

- Large-Scale & Pretraining Datasets (ViT-focused)

Dataset	Purpose	Scale	Notes	Official Link
JFT-300M	Pretraining	300M images	Internal Google dataset (ViT)	Not public
OpenImages	Classification / Detection	9M+ images	Large-scale, noisy labels	https://storage.googleapis.com/openimages/web/index.html
YFCC100M	SSL / VLM	100M images	Flickr-based, weak labels	https://multimediacommons.wordpress.com/yfcc100m-core-dataset/
LAION-400M	Vision-Language	400M pairs	CLIP-style training	https://laion.ai/blog/laion-400-open-dataset/
LAION-5B	Vision-Language	5B pairs	Foundation model scale	https://laion.ai/blog/laion-5b/

Image dataset

- Detection / Segmentation Benchmarks

Dataset	Task	#Classes	#Images	Typical Usage	Official Link
PASCAL VOC	Detection / Segmentation	20	~11K	Classical benchmark	http://host.robots.ox.ac.uk/pascal/VOC/
MS COCO	Detection / Seg / Keypoints	80	330K	Standard detection benchmark	https://cocodataset.org/
Cityscapes	Segmentation	19	25K	Autonomous driving	https://www.cityscapes-dataset.com/
ADE20K	Segmentation	150	25K	Complex scenes	https://groups.csail.mit.edu/vision/datasets/ADE20K/
LVIS	Detection	1,200+	164K	Long-tail detection	https://www.lvisdataset.org/